Práctica: π con Leibniz, sockets y AWS — guía línea por línea

Oct 14, 2026 · @Julián Andrés Foglia Wilches

Cómo usar este documento

Este documento reconstruye, paso a paso, el programa que calcula π con dos procesos no emparentados que se comunican por sockets TCP, primero en tu Ubuntu y después en dos instancias EC2 de AWS. Cada archivo se explica línea por línea: qué hace cada variable, cada función y cada concepto que aparece.

Los pasos 2 y 3 muestran las versiones mínimas del servidor y del cliente, que sirven para aprender la API de sockets. Los pasos 4 y 5 construyen la versión final, que es la que debes entregar; está completa al final del documento. Cuando un concepto ya se explicó, los pasos siguientes solo explican lo nuevo.

Antes de los pasos está el glosario. Vuelve a él cada vez que aparezca una sigla que no recuerdes.

Glosario de siglas y nombres

Programación en C

Sigla o nombre	Viene de	Qué es
`argc` / `argv`	*argument count* / *argument vector*	Cuántas palabras se escribieron en la terminal al lanzar el programa, y el arreglo con esas palabras como texto.
`atoi`	*ASCII to integer*	Convierte texto en un `int`. No detecta errores: con `"abc"` devuelve 0.
`strtoll`	*string to long long*	Convierte texto en un entero de 64 bits y puede decirte dónde dejó de convertir.
`printf` / `fprintf`	*print formatted* / *file print formatted*	Escriben texto con formato; `fprintf` permite elegir a dónde (por ejemplo `stderr`).

Sigla o nombre	Viene de	Qué es
`stdin` / `stdout` / `stderr`	*standard input / output / error*	Entrada estándar (descriptor 0), salida estándar (1) y salida de errores (2).
`perror`	*print error*	Imprime tu texto seguido de la descripción del error guardado en `errno`.
`memset` / `memcpy`	*memory set* / *memory copy*	Llenan un bloque de memoria con un valor / copian bytes de un lugar a otro.
`size_t` / `ssize_t`	*size type* / *signed size type*	Enteros para cantidades de bytes; `ssize_t` tiene signo para poder valer -1 (error).
`int64_t` / `uint64_t`	*integer 64-bit type* / *unsigned …*	Enteros de exactamente 64 bits, con y sin signo, en cualquier máquina.
`socklen_t`	*socket length type*	Tipo para el tamaño de una estructura de dirección.
`errno`	*error number*	Variable que guarda el código del último error de una llamada al sistema.
`EXIT_FAILURE`	—	Constante (vale 1) que indica que el programa terminó con error.
`NaN`	*Not a Number*	Valor de punto flotante para resultados sin sentido, como 0/0.
`IEEE 754`	*Institute of Electrical and Electronics Engineers*, norma 754	El estándar que define cómo se guardan los `double` en bits.
GCC	*GNU Compiler Collection*	El compilador. GNU es un acrónimo recursivo: *GNU's Not Unix*.
glibc	*GNU C Library*	La biblioteca estándar de C en Linux: ahí viven `printf`, `recv`, `send`, etc.

Sigla o nombre	Viene de	Qué es
API	*Application Programming Interface*	Conjunto de funciones que una biblioteca ofrece a tus programas.
POSIX	*Portable Operating System Interface* (la X por Unix)	Estándar que define llamadas como `socket`, `send` o `close`, comunes a los sistemas tipo Unix.

Llamadas al sistema y sockets

Sigla o nombre	Viene de	Qué es
fd	*file descriptor*	Descriptor de archivo: el número entero con el que un proceso se refiere a algo abierto (archivo, pipe, socket).
`AF_INET`	*Address Family: Internet*	Familia de direcciones IPv4.
`SOCK_STREAM`	*socket stream*	Socket de flujo de bytes confiable; en IPv4 equivale a TCP.
`IPPROTO_IP`	*IP protocol*	Valor 0 del argumento de protocolo de `socket`: "el protocolo por defecto".
`SOL_SOCKET`	*socket option level: socket*	Indica que la opción que se configura pertenece al nivel del socket en general.
`SO_REUSEADDR`	*socket option: reuse address*	Permite volver a hacer `bind` a un puerto que quedó en `TIME_WAIT`.
`INADDR_ANY`	*Internet address: any*	La dirección 0.0.0.0: escuchar en todas las interfaces de red.
`htons` / `htonl`	*host to network short / long*	Convierten un entero de 16 / 32 bits al orden de bytes de la red.
`ntohs`	*network to host short*	El inverso de `htons`.
`htobe64` / `be64toh`	*host to big-endian 64 / big-endian 64 to host*	Lo mismo para 64 bits.

Sigla o nombre	Viene de	Qué es
`inet_pton` / `inet_ntop`	*presentation to network* / *network to presentation*	Convierten una IP entre texto (`"172.31.0.27"`) y binario.
`MSG_NOSIGNAL`	*message flag: no signal*	Opción de `send` para que el kernel no envíe `SIGPIPE`.
`EADDRINUSE`	*error: address in use*	El puerto ya está ocupado.
`EINTR`	*error: interrupted*	Una señal interrumpió la llamada; se debe reintentar.
`EPIPE`	*error: broken pipe*	Se intentó escribir en una conexión que el otro extremo cerró.
`EINPROGRESS`	*error: operation in progress*	La conexión seguía en curso cuando se agotó el tiempo.
`SIGINT`	*signal: interrupt*	Señal que envía `Ctrl+C`; por defecto termina el proceso.
`SIGTERM`	*signal: terminate*	Señal que envía `kill`; por defecto termina el proceso.
`SIGPIPE`	*signal: broken pipe*	Señal que mata en silencio a quien escribe en una conexión rota.
EOF	*End Of File*	Fin de archivo. En un socket, `recv` lo indica devolviendo 0.
vDSO	*virtual Dynamic Shared Object*	Código del kernel mapeado en cada proceso para que funciones como `clock_gettime` no necesiten cambiar a modo kernel.

Redes

Sigla o nombre	Viene de	Qué es
IP / IPv4	*Internet Protocol*, versión 4	Protocolo que lleva paquetes entre máquinas; una dirección IPv4 tiene 32 bits.
TCP	*Transmission Control Protocol*	Protocolo que entrega un flujo de bytes completo, en orden y sin duplicados.

Sigla o nombre	Viene de	Qué es
SYN / FIN / RST	*synchronize* / *finish* / *reset*	Marcas de los paquetes TCP: iniciar conexión, cerrarla de forma ordenada, rechazarla o abortarla.
`TIME_WAIT`	—	Estado en que TCP deja un puerto un rato después de cerrar, por si llegan paquetes tardíos.
NAT	*Network Address Translation*	Traducción de direcciones: AWS traduce la IP pública a la privada de tu instancia.
CIDR	*Classless Inter-Domain Routing*	Notación `IP/bits` para un bloque de direcciones; `/32` es una sola dirección.
Big-endian / little-endian	—	Orden de bytes: el más significativo primero / el menos significativo primero.

AWS y herramientas de Linux

Sigla o nombre	Viene de	Qué es
AWS	*Amazon Web Services*	La nube de Amazon.
EC2	*Elastic Compute Cloud*	Servicio de AWS para alquilar máquinas virtuales, llamadas instancias.
AMI	*Amazon Machine Image*	Plantilla con el sistema operativo con que nace una instancia.
LTS	*Long Term Support*	Versión de Ubuntu con soporte extendido (24.04 LTS).
VPC	*Virtual Private Cloud*	Tu red privada dentro de AWS, donde viven las IP privadas.
vCPU	*virtual CPU*	Una CPU lógica asignada a una máquina virtual.
CPU	*Central Processing Unit*	Procesador.
x86 / ARM	familia Intel 8086 / *Advanced RISC Machines*	Dos arquitecturas de procesador; tu portátil y la t3.micro son x86 de 64 bits.

Sigla o nombre	Viene de	Qué es
SSH	*Secure Shell*	Protocolo para controlar otra máquina por terminal de forma cifrada.
`scp`	*secure copy*	Copia archivos entre máquinas a través de SSH.
RSA	*Rivest–Shamir–Adleman* (sus inventores)	Algoritmo de criptografía de la llave `so-pi.pem`.
`.pem`	*Privacy-Enhanced Mail*	Formato de texto del archivo de la llave privada.
PID	*Process Identifier*	Número único de cada proceso.
`ps`	*process status*	Muestra los procesos y su estado.
`STAT` / `PSR` / `WCHAN`	*state* / *processor* / *wait channel*	Columnas de `ps`: estado (`R` ejecutando, `S` dormido), CPU donde corrió por última vez y función del kernel donde espera.
`ss`	*socket statistics*	Lista los sockets del sistema.
`nc`	*netcat*	Abre conexiones TCP desde la terminal, como cliente o como servidor.
`strace`	*system call trace*	Muestra cada llamada al sistema que hace un proceso.
`sudo`	*superuser do*	Ejecuta un comando como administrador.
`apt`	*Advanced Package Tool*	Instalador de programas de Ubuntu.
`chmod`	*change mode*	Cambia los permisos de un archivo.
`curl`	*client URL*	Hace peticiones web desde la terminal.
`-Wall` / `-Wextra` / `-O2` / `-lm`	*warnings all*, *warnings extra*, *optimization level 2*, *link libm*	Opciones de `gcc`: advertencias, optimización y enlazar la biblioteca matemática.

Visión general

El programa reparte el cálculo de π entre dos procesos independientes: un **coordinador** (servidor) y un **trabajador** (cliente). Cada uno suma la mitad de los términos de la serie de Leibniz y se comunican por un socket TCP. En AWS cada proceso corre en su propia máquina, y el resultado fue tardar la mitad que un solo proceso.

Supuestos de diseño

No había enunciado oficial, así que la práctica se construyó con estos supuestos explícitos:

1. **Lenguaje C**, con la API de sockets de POSIX.
2. **Protocolo TCP** (`SOCK_STREAM`): los bytes llegan completos y en orden.
3. **Dos procesos**: el servidor recibe N (número de términos), se queda con la primera mitad y envía la segunda al cliente.
4. **Ambos calculan al mismo tiempo**, cada uno en su propia instancia EC2.

Procesos emparentados y no emparentados

Esta es la idea que justifica usar sockets.

- **Procesos emparentados** nacen con `fork()`. El hijo hereda una copia de la tabla de descriptores del padre, así que un `pipe()` creado antes del `fork()` ya los conecta. El punto de encuentro se hereda.
- **Procesos no emparentados** se lanzan por separado (`./servidor` en una terminal, `./cliente` en otra). No heredan nada, así que necesitan un **nombre que ambos conozcan de antemano**. En la Práctica 1 era la ruta de un FIFO en el sistema de archivos. Aquí, como están en máquinas distintas que no comparten sistema de archivos, el nombre es una **dirección IP más un puerto**.

Analogía: un `pipe` es pasarle una nota a alguien de tu misma casa; un socket es llamar por teléfono a alguien de otra ciudad, para lo cual necesitas su número (la IP) y su extensión (el puerto).

Qué hace cada proceso, en orden

1. El servidor crea un socket, lo asocia al puerto 5000 y espera (`socket`, `bind`, `listen`, `accept`).
2. El cliente crea su socket y se conecta a la IP y el puerto del servidor (`socket`, `connect`).
3. El servidor envía al cliente su rango de términos: `[N/2, N)`.
4. **Al mismo tiempo**, el servidor calcula `[0, N/2)` y el cliente calcula `[N/2, N)`.

5. El cliente devuelve su suma parcial y cierra.
6. El servidor recibe esa suma, calcula $\pi = 4 \times (\text{suma}_a + \text{suma_b})$, lo imprime y cierra.

El orden del paso 3 antes del 4 es una decisión de diseño: si el servidor calculara su mitad antes de enviar el rango, el cliente estaría bloqueado esperando y los dos cálculos ocurrirían uno detrás del otro, sin ninguna ganancia de tiempo.

Archivos de la práctica

Archivo	Responsabilidad
`leibniz.c`	Versión de un solo proceso; sirve de referencia y de línea base de tiempo.
`serie.h`	Matemáticas: la suma parcial de Leibniz y una función para medir tiempo.
`protocolo.h`	Red: el contrato de mensajes y las funciones para enviar y recibir números.
`servidor.c`	El coordinador.
`cliente.c`	El trabajador.
`prueba_bytes.c`	Programa auxiliar para ver el orden de bytes en memoria.

Paso 1: la serie de Leibniz en un solo proceso

El primer paso resuelve el problema matemático sin red: `leibniz.c` calcula π sumando dos mitades de la serie dentro del mismo proceso, para comprobar que el reparto es correcto antes de distribuirlo.

La serie

```
\frac{\pi}{4} = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots
```

Cada término depende **solo de su índice k**, no del término anterior. Esa propiedad permite partir la suma en rangos, calcularlos en procesos o máquinas distintas y sumar los resultados al final. Si cada término dependiera del anterior, no se podría repartir.

El código: `leibniz.c`

```
#include <stdio.h>    // printf, fprintf
#include <stdlib.h>   // strtoll
#include <stdint.h>   // int64_t

double suma_parcial(int64_t inicio, int64_t fin) {
    double suma = 0.0;
    for (int64_t k = inicio; k < fin; k++) {
        double signo = (k % 2 == 0) ? 1.0 : -1.0;
        suma += signo / (2.0 * k + 1.0);
    }
    return suma;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Uso: %s <num_terminos>\n", argv[0]);
        return 1;
    }

    int64_t n = strtoll(argv[1], NULL, 10);
    int64_t mitad = n / 2;

    double suma_a = suma_parcial(0, mitad);   // lo que hará el servidor
    double suma_b = suma_parcial(mitad, n);   // lo que hará el cliente

    double pi = 4.0 * (suma_a + suma_b);
    printf("pi ~= %.15f\n", pi);
    return 0;
}
```

Línea por línea

Las cabeceras (`#include`)

- `#include` es una orden para el **preprocesador**, la etapa que corre antes de compilar: pega el contenido del archivo indicado en ese lugar. Los ángulos `< >` indican que es una cabecera del sistema.
- `stdio.h` (*standard input/output*): declara `printf` y `fprintf`.
- `stdlib.h` (*standard library*): declara `strtoll`.
- `stdint.h` (*standard integers*): declara `int64_t`, un entero de **exactamente 64 bits** en cualquier máquina. Con `long` no hay esa garantía: mide 8 bytes en Linux de 64 bits y 4 en Windows. Esto será crucial en el paso 4, porque al enviar un número por la red los

dos lados deben estar de acuerdo en cuántos bytes ocupa.

`double suma_parcial(int64_t inicio, int64_t fin)`

- Declara una función que recibe dos enteros de 64 bits y devuelve un `double` (número de punto flotante de 64 bits, doble precisión).
- Suma los términos con k desde `inicio` hasta `fin - 1`. El rango es **semiabierto**, `[inicio, fin)`: incluye el inicio y excluye el fin. Así, dos rangos consecutivos `[0, mitad)` y `[mitad, n)` se tocan sin repetir ni saltarse ningún término.
- `double suma = 0.0;` crea el acumulador en cero. En C las variables locales **no se inicializan solas**: sin el `= 0.0` contendría lo que hubiera en esa posición de la pila (basura).
- `for (int64_t k = inicio; k < fin; k++)` recorre los índices. Sus tres partes: valor inicial, condición para seguir, y `k++` (sumar 1 al final de cada vuelta).
- `double signo = (k % 2 == 0) ? 1.0 : -1.0;` usa dos operadores. `%` es el **residuo** de la división: `k % 2` vale 0 si k es par. `? :` es el **operador ternario**: "si la condición es cierta, el primer valor; si no, el segundo". Reemplaza a (-1)^k sin calcular una potencia en cada vuelta.
- `suma += signo / (2.0 * k + 1.0);` suma el término al acumulador (`a += b` es `a = a + b`). **Punto clave:** si se escribiera `1 / (2 * k + 1)`, ambos operandos serían enteros y C haría **división entera**: `1/3` da 0 y π saldría 4. Al escribir `2.0` la operación se hace en `double`. Además se evita que `2 * k` desborde el entero con k muy grandes.
- `return suma;` devuelve el resultado a quien llamó.

`int main(int argc, char *argv[])`

- `main` es donde empieza todo programa en C. Devuelve un `int` al sistema operativo.
- `argc` (*argument count*) es cuántas palabras se escribieron en la terminal. `argv` (*argument vector*) es un arreglo de cadenas con esas palabras. Con `./leibniz 1000000`, `argc` vale 2, `argv[0]` es `"./leibniz"` y `argv[1]` es `"1000000"`.
- `if (argc != 2)`: si el usuario no pasó exactamente un argumento, se imprime cómo usar el programa. `fprintf(stderr, ...)` escribe en la **salida de errores**, no en la normal, para que los mensajes de error no se mezclen con los resultados.
- `%s` dentro del texto es un **especificador de formato**: se reemplaza por la cadena `argv[0]`. `\n` es el salto de línea.
- `return 1;` termina con código de error. **Convención de Linux:** 0 es éxito; cualquier otro valor, error. Se puede ver con `echo $?` justo después de ejecutar.

- `int64_t n = strtoll(argv[1], NULL, 10);` convierte el texto en número. `strtoll` (*string to long long*) recibe la cadena, un puntero opcional para saber dónde se detuvo (aquí `NULL`, "no me interesa") y la base numérica (10, decimal).
- `int64_t mitad = n / 2;` aquí sí queremos división entera.
- Las dos llamadas a `suma_parcial` **simulan en un solo proceso lo que luego harán dos**. Si el resultado es correcto, el reparto es correcto.
- `double pi = 4.0 * (suma_a + suma_b);` multiplica por 4 porque la serie converge a π/4.
- `printf("pi ~= %.15f\n", pi);` imprime π. `%.15f` significa: número de punto flotante con 15 decimales.
- `return 0;` termina con éxito.

Compilar y ejecutar

```
gcc -Wall -Wextra -O2 -o leibniz leibniz.c
./leibniz 1000000
time ./leibniz 1000000000
```

- `gcc` compila el archivo `.c` y produce un ejecutable.
- `-Wall -Wextra`: activa casi todas las advertencias. Un programa de entrega no debería compilar con advertencias.
- `-O2`: nivel 2 de optimización. Importa porque vamos a medir tiempo.
- `-o leibniz`: nombre del ejecutable de salida (*output*).
- `./leibniz`: el `./` indica "el programa que está en esta carpeta".
- `time` ejecuta el programa y al final informa cuánto tardó.

Lo que se observó

N	Resultado	Diferencia con π
10^6	3.141591653589692	$\approx 10^{-6}$
10^9	3.141592652589258	$\approx 10^{-9}$
10^{10}	3.141592653488201	$\approx 1{,}016 \times 10^{-10}$

- **El error es casi exactamente 1/N.** Con 1000 veces más términos solo se ganan 3 dígitos: la serie converge muy lento. Por eso conviene repartirla: mucho trabajo de CPU y muy poca información que intercambiar.

- **Con N par la aproximación queda por debajo de π**, porque el último término sumado es negativo.
- **Error de redondeo.** Con 10^9 términos aparecen diferencias en los últimos dígitos que no se explican solo por cortar la serie: cada una de las mil millones de sumas en `double` redondea un poco. La suma en punto flotante **no es asociativa**: `(a + b) + c` puede diferir de `a + (b + c)` en el último bit. Por eso, si se cambia el orden de las sumas, los últimos dígitos pueden cambiar; si se repiten las mismas operaciones en el mismo orden, el resultado es idéntico bit a bit.

Lo que dice `time`

Con N = 10^9 se obtuvo `0,88s user 0,00s system 99% cpu 0,877 total`.

- **user**: tiempo de CPU en **modo usuario**, ejecutando tu ciclo `for`.
- **system**: tiempo de CPU en **modo kernel**. Casi cero, porque el programa solo hace llamadas al sistema al arrancar, al imprimir (`printf` termina en la llamada `write`) y al salir.
- **cpu 99 %**: el proceso estuvo casi siempre en estado **ejecutando**, nunca bloqueado. Es un proceso **limitado por CPU** (*CPU-bound*).
- **total**: tiempo de reloj de pared.

Con N = 10^{10} tardó 8,83 s, casi exactamente 10 veces más: el costo por término es constante. Es la línea base contra la que se compara la versión distribuida.

Paso 2: el servidor mínimo

El servidor mínimo abre el puerto 5000, espera a un cliente, le envía un saludo y termina. Sirve para aprender las cuatro llamadas que convierten un proceso en servidor: `socket`, `bind`, `listen` y `accept`.

Concepto previo: el descriptor de archivo

Cuando un proceso abre algo (un archivo, un pipe, un FIFO, un socket), el kernel lo registra en la **tabla de archivos abiertos de ese proceso** y le devuelve un número entero: el **descriptor de archivo** (fd). Todo proceso nace con tres: 0 (`stdin`), 1 (`stdout`) y 2 (`stderr`). Por eso el primer socket que crees recibe el número 3, el primero libre.

Analogía: el fd es la ficha de un guardarropa. Tú no tienes el abrigo, solo el número; cada vez que lo necesitas le das la ficha al encargado (el kernel) y él hace el trabajo. Por eso `send`, `recv` y `close` reciben un `int`: es la ficha.

Las cuatro llamadas del servidor

Llamada	Qué hace	Analogía
`socket()`	Crea el extremo de comunicación y devuelve su fd	Comprar un teléfono
`bind()`	Le asigna una dirección: IP y puerto	Asignarle un número
`listen()`	Lo marca como pasivo: acepta llamadas entrantes	Activar el timbre
`accept()`	Espera una conexión y devuelve un **fd nuevo** para hablar con ese cliente	Contestar por una línea nueva, mientras la principal sigue disponible

Lo más importante: hay **dos sockets distintos**. El de escucha solo recibe conexiones nuevas; el que devuelve `accept` es por donde realmente se habla con el cliente.

El código: `servidor.c` (versión mínima)

```
#include <stdio.h>       // printf, perror
#include <stdlib.h>      // exit, EXIT_FAILURE
#include <string.h>      // memset, strlen
#include <unistd.h>      // close
#include <arpa/inet.h>   // sockaddr_in, htons, htonl, ntohs, inet_ntop
#include <sys/socket.h>  // socket, setsockopt, bind, listen, accept, send

#define PUERTO 5000

int main(void) {
    // 1. Crear el socket de escucha
    int fd_escucha = socket(AF_INET, SOCK_STREAM, 0);
    if (fd_escucha < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }

    // 2. Permitir reutilizar el puerto de inmediato
    int si = 1;
    if (setsockopt(fd_escucha, SOL_SOCKET, SO_REUSEADDR, &si, sizeof(si)) <
0) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }

    // 3. Construir la dirección donde vamos a escuchar
```

```
    struct sockaddr_in dir;
    memset(&dir, 0, sizeof(dir));
    dir.sin_family = AF_INET;
    dir.sin_addr.s_addr = htonl(INADDR_ANY);
    dir.sin_port = htons(PUERTO);

    // 4. Asociar el socket a esa dirección
    if (bind(fd_escucha, (struct sockaddr *)&dir, sizeof(dir)) < 0) {
        perror("bind");
        exit(EXIT_FAILURE);
    }

    // 5. Convertirlo en socket pasivo
    if (listen(fd_escucha, 1) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }
    printf("Servidor escuchando en el puerto %d...\n", PUERTO);

    // 6. Esperar a un cliente (aquí el proceso se bloquea)
    struct sockaddr_in dir_cliente;
    socklen_t tam = sizeof(dir_cliente);
    int fd_cliente = accept(fd_escucha, (struct sockaddr *)&dir_cliente,
&tam);
    if (fd_cliente < 0) {
        perror("accept");
        exit(EXIT_FAILURE);
    }

    char ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &dir_cliente.sin_addr, ip, sizeof(ip));
    printf("Cliente conectado desde %s:%d\n", ip,
ntohs(dir_cliente.sin_port));

    // 7. Enviar un saludo por el socket conectado
    const char *msg = "Hola desde el servidor\n";
    send(fd_cliente, msg, strlen(msg), 0);

    // 8. Liberar ambos descriptores
    close(fd_cliente);
    close(fd_escucha);
    return 0;
}
```

Línea por línea

Cabeceras nuevas

- `string.h`: `memset` (llenar memoria) y `strlen` (*string length*, largo de una cadena sin contar el terminador).
- `unistd.h` (*Unix standard*): `close`, la llamada que libera un descriptor.
- `arpa/inet.h` (*ARPA Internet*; ARPANET fue la red precursora de internet): estructuras de direcciones IPv4 y funciones de conversión (`htons`, `inet_ntop`, etc.).
- `sys/socket.h`: las llamadas de sockets (`socket`, `bind`, `listen`, `accept`, `send`).

`#define PUERTO 5000` crea una constante del preprocesador: antes de compilar, cada aparición de `PUERTO` se reemplaza por `5000`. Un puerto es un número de 16 bits (0 a 65535) que identifica a qué programa de la máquina va dirigida una conexión.

`int main(void)`: `void` indica que este programa no usa argumentos.

Patrón de manejo de errores. Casi todas las llamadas al sistema devuelven -1 si fallan y guardan el motivo en la variable global `errno`. `perror("bind")` imprime tu texto seguido de la descripción correspondiente, por ejemplo `bind: Address already in use`. `exit(EXIT_FAILURE)` termina el proceso con código de error. Este patrón se repite en cada llamada; ignorar errores de llamadas al sistema es un punto débil típico en una revisión.

1. `int fd_escucha = socket(AF_INET, SOCK_STREAM, 0);`

- `AF_INET`: familia de direcciones IPv4.
- `SOCK_STREAM`: flujo de bytes confiable y ordenado; con IPv4 significa TCP.
- `0`: el protocolo por defecto para esa combinación, es decir TCP.
- Devuelve el fd del nuevo socket, que todavía no tiene dirección ni conexión.

2. `setsockopt(fd_escucha, SOL_SOCKET, SO_REUSEADDR, &si, sizeof(si))`

- Configura una opción del socket. Recibe: el socket, el nivel de la opción (`SOL_SOCKET`, opciones generales), la opción (`SO_REUSEADDR`), un puntero al valor (`&si`, la dirección de la variable que vale 1, "activado") y el tamaño de ese valor.
- `&` es el operador **dirección de**: entrega la posición en memoria de la variable, para que la función pueda leerla.
- `sizeof(si)` devuelve cuántos bytes ocupa la variable (4 para un `int`).
- **Por qué hace falta:** al cerrar una conexión, TCP deja el puerto en el estado `TIME_WAIT` hasta un minuto, por si llegan paquetes tardíos. Sin esta opción, relanzar el servidor enseguida falla con `Address already in use`.

3. La dirección: `struct sockaddr_in dir;`

- Un `struct` es un tipo que agrupa varios campos. `sockaddr_in` (*socket address, Internet*) describe una dirección IPv4.
- `memset(&dir, 0, sizeof(dir));` pone todos sus bytes en cero. La estructura nace con basura y tiene campos de relleno que el kernel espera en cero.
- `dir.sin_family = AF_INET;` el punto `.` accede a un campo del struct. Repite que es IPv4.
- `dir.sin_addr.s_addr = htonl(INADDR_ANY);` `INADDR_ANY` es la dirección 0.0.0.0, que significa "escucha en **todas** las interfaces de red de esta máquina". Es clave en AWS: una instancia no ve su IP pública en ninguna interfaz, solo la privada, así que hacer `bind` a la pública fallaría.
- `dir.sin_port = htons(PUERTO);` aquí aparece el **orden de bytes**. Tu procesador x86 es *little-endian*: guarda primero el byte menos significativo. Los protocolos de red usan *big-endian*. El puerto 5000 es `0x1388`; en memoria queda `88 13`, pero la red espera `13 88`. `htons` (*host to network short*) hace esa inversión para 16 bits; `htonl` para 32 bits. En una máquina big-endian no hacen nada, por eso se usan siempre.

4. **`bind(fd_escucha, (struct sockaddr *)&dir, sizeof(dir))`**

- Asocia el socket a la dirección.
- `(struct sockaddr *)` es un ***cast*** (conversión de tipo). `bind` sirve para muchas familias de direcciones (IPv4, IPv6, sockets locales), cada una con su propia estructura. Recibe un tipo genérico más el tamaño, y el kernel mira `sin_family` para saber cómo interpretarlo. Es la forma que tiene C de imitar el polimorfismo.

5. **`listen(fd_escucha, 1)`**

- Marca el socket como pasivo: ya no sirve para conectarse a otros, solo para recibir conexiones.
- El `1` es el ***backlog***: cuántas conexiones pueden esperar en cola antes de que se llame a `accept`.
- El saludo inicial de TCP (*three-way handshake*: SYN, SYN-ACK, ACK) lo completa **el kernel**, no tu programa. La conexión puede quedar establecida en la cola antes de que tu proceso llame a `accept`.

6. **`accept(fd_escucha, (struct sockaddr *)&dir_cliente, &tam)`**

- Si no hay conexiones en la cola, el kernel pasa el proceso al estado **bloqueado**: deja de usar CPU hasta que llegue un cliente.
- Devuelve `fd_cliente`, el socket conectado, y llena `dir_cliente` con la dirección del cliente.
- `socklen_t tam` se pasa por puntero porque es un parámetro de **entrada y salida**: le dices al kernel cuánto espacio tienes y él escribe cuánto usó.

- `char ip[INET_ADDRSTRLEN];` crea un arreglo de caracteres del tamaño justo para una IPv4 en texto (16 bytes, incluido el terminador).
- `inet_ntop` (*network to presentation*) convierte la IP de binario a texto, como `"127.0.0.1"`. `ntohs` (*network to host short*) deshace el `htons` para imprimir el puerto. `%d` imprime un entero.

7. **`send(fd_cliente, msg, strlen(msg), 0)`**

- `const char *msg` es un puntero a una cadena que no se modificará (`const`).
- `send` envía los bytes por el socket **conectado**, no por el de escucha. Recibe: socket, datos, cantidad de bytes y opciones (0, ninguna). En esta versión se ignora lo que devuelve; el paso 4 explica por qué eso es un problema.

8. **`close(...)`** libera ambas entradas de la tabla de descriptores.

Pruebas y lo que se observó

- Al ejecutar `./servidor` se imprime el mensaje y el proceso queda quieto: está **bloqueado en `accept`**.
- `ss -tlnp | grep 5000` muestra un socket en estado `LISTEN` en `0.0.0.0:5000` (`t` TCP, `l` solo los que escuchan, `n` números, `p` qué proceso lo tiene).
- `ps -o pid,stat,wchan:25,cmd -C servidor` muestra `S` (*sleeping*, dormido esperando un evento) y en `WCHAN` la función del kernel donde espera.
- `ls -l /proc/<pid>/fd` muestra los descriptores 0, 1 y 2 apuntando a la terminal, y el 3 apuntando a `socket:[número]`. `/proc` es un sistema de archivos virtual con información de cada proceso.
- Al conectarse con `nc 127.0.0.1 5000`, el servidor imprimió `Cliente conectado desde 127.0.0.1:44536`. **Ese puerto lo eligió el kernel**: es un **puerto efímero** del rango de `/proc/sys/net/ipv4/ip_local_port_range` (en Ubuntu, de 32768 a 60999). Toda conexión TCP se identifica por cuatro valores: IP y puerto de origen, IP y puerto de destino.
- El servidor vivió 65 segundos, casi todos bloqueado sin usar CPU: un proceso **limitado por E/S**, lo opuesto al 99 % de CPU de `leibniz`.

El error `Address already in use` que apareció después

Días después, `bind` falló con ese error aunque el código tenía `SO_REUSEADDR`. El mismo código de error (`EADDRINUSE`) tiene dos causas posibles:

Causa	¿La resuelve `SO_REUSEADDR`?
El puerto quedó en `TIME_WAIT` tras cerrar una conexión reciente	Sí
Otro socket está escuchando activamente en ese puerto	No

`SO_REUSEADDR` permite reutilizar un puerto "en duelo", nunca quitarle el puerto a un proceso vivo. Se diagnostica con `ss -tlnp | grep 5000` (o con `sudo` si el proceso es de otro usuario), y se resuelve terminando ese proceso con `kill <PID>`, que le envía la señal `SIGTERM`. Al morir un proceso, el kernel cierra todos sus descriptores y el puerto queda libre.

Paso 3: el cliente mínimo

El cliente mínimo se conecta al servidor, lee todo lo que llegue hasta que el servidor cierre y termina. Es más corto porque su papel es **activo**: no espera a nadie, sale a buscar al otro.

Servidor (pasivo)	Cliente (activo)
`socket()`	`socket()`
`bind()` a un puerto conocido	no hace `bind`
`listen()`	—
`accept()`, que devuelve un fd nuevo	`connect()`, que usa el mismo fd
`send` / `recv`	`send` / `recv`
`close()`	`close()`

El cliente no hace `bind` porque a nadie le importa en qué puerto está: el servidor no lo va a buscar, solo le responde. Al llamar a `connect`, el kernel le asigna automáticamente un puerto efímero.

El código: `cliente.c` (versión mínima)

```
#include <stdio.h>       // printf, fprintf, perror
#include <stdlib.h>      // exit, EXIT_FAILURE, atoi
#include <string.h>      // memset
#include <unistd.h>      // close
#include <arpa/inet.h>   // sockaddr_in, htons, inet_pton
```

```
#include <sys/socket.h>  // socket, connect, recv

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Uso: %s <ip_servidor> <puerto>\n", argv[0]);
        return 1;
    }

    // 1. Crear el socket
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }

    // 2. Construir la dirección DEL SERVIDOR
    struct sockaddr_in dir;
    memset(&dir, 0, sizeof(dir));
    dir.sin_family = AF_INET;
    dir.sin_port = htons(atoi(argv[2]));
    if (inet_pton(AF_INET, argv[1], &dir.sin_addr) != 1) {
        fprintf(stderr, "IP invalida: %s\n", argv[1]);
        exit(EXIT_FAILURE);
    }

    // 3. Conectarse (aquí ocurre el saludo de TCP)
    if (connect(fd, (struct sockaddr *)&dir, sizeof(dir)) < 0) {
        perror("connect");
        exit(EXIT_FAILURE);
    }
    printf("Conectado a %s:%s\n", argv[1], argv[2]);

    // 4. Leer hasta que el servidor cierre la conexión
    char buf[256];
    ssize_t n;
    while ((n = recv(fd, buf, sizeof(buf) - 1, 0)) > 0) {
        buf[n] = '\0';
        printf("Recibido (%zd bytes): %s", n, buf);
    }
    if (n < 0) {
        perror("recv");
    } else {
        printf("El servidor cerro la conexion\n");
    }
```

```
    // 5. Liberar el descriptor
    close(fd);
    return 0;
}
```

Línea por línea

Argumentos `<ip_servidor> <puerto>`. La IP no va fija en el código porque en AWS será la IP privada de la otra instancia, que no se conoce hasta crearla. Así el mismo ejecutable sirve en local (`127.0.0.1`) y en la nube sin recompilar. `127.0.0.1` es la dirección de ***loopback***: siempre significa "esta misma máquina".

2. La dirección. La estructura es la misma que en el servidor, pero cambia su significado: allá `dir` era **dónde escucho yo**; aquí es **a dónde me quiero conectar**.

- `htons(atoi(argv[2]))`: `atoi` (*ASCII to integer*) convierte el texto `"5000"` en entero. Es la versión simple de `strtoll` y no detecta errores. `htons` lo pasa a orden de red.
- `inet_pton(AF_INET, argv[1], &dir.sin_addr)` (*presentation to network*), el inverso de `inet_ntop`: convierte el texto `"127.0.0.1"` en los 4 bytes binarios de la dirección. Devuelve 1 si funcionó y 0 si el texto no era una IPv4 válida. No resuelve nombres como `google.com`.

3. **`connect(fd, (struct sockaddr *)&dir, sizeof(dir))`.** El kernel envía el primer paquete del saludo de TCP y el proceso **se bloquea** hasta recibir respuesta. Es el equivalente activo de `accept`: cuando regresa con éxito, la conexión ya está establecida en ambos extremos.

4. El ciclo de `recv`. Aquí hay tres ideas nuevas.

- `char buf[256];` reserva 256 bytes en la pila para recibir datos (un ***buffer***).
- `recv(fd, buf, sizeof(buf) - 1, 0)` copia en `buf` los bytes que hayan llegado, como máximo 255, y devuelve **cuántos leyó**. Ese número tiene tres significados:
  - mayor que 0: llegaron datos;
  - 0: el otro extremo **cerró la conexión** de forma ordenada; es el "fin de archivo" (EOF) del socket;
  - -1: hubo un error.
- `ssize_t` es un entero **con signo** del tamaño adecuado para cantidades de bytes; tiene signo justamente para poder valer -1. Se imprime con `%zd`.
- `while ((n = recv(...)) > 0)` hace dos cosas: guarda en `n` lo que devuelve `recv` y luego compara `n > 0`. El ciclo sigue mientras lleguen datos.

- `buf[n] = '\0';` `recv` copia bytes crudos y no sabe nada de cadenas de C. Una cadena en C termina con el byte `'\0'`; para poder imprimir con `%s` hay que agregarlo a mano, y por eso se dejó un byte libre con `sizeof(buf) - 1`.
- Al salir del ciclo, `n < 0` indica error y `n == 0` indica cierre ordenado.

¿Por qué un ciclo y no un solo `recv`? Esta es la idea más importante del paso: **TCP es un flujo de bytes, no de mensajes**. No hay garantía de que un `send` llegue en un solo `recv`. Un envío de 23 bytes puede llegar como 10 + 13, y dos envíos seguidos pueden llegar juntos. Analogía: TCP es una tubería de agua, no un servicio de paquetes; lo que entra sale en orden, pero el agua no recuerda en qué baldes la echaste.

Pruebas y lo que se observó

Prueba	Resultado	Por qué
Servidor encendido, `./cliente 127.0.0.1 5000`	`Recibido (23 bytes): Hola desde el servidor` y luego `El servidor cerro la conexion`	23 bytes = 22 caracteres + el salto de línea. El mensaje final aparece porque el servidor hizo `close` y el siguiente `recv` devolvió 0.
Servidor apagado	`connect: Connection refused`	El kernel de la máquina destino recibió el intento, vio que nadie escucha en ese puerto y respondió de inmediato con un rechazo (un paquete RST).
`./cliente hola 5000`	`IP invalida: hola`	Falla `inet_pton` antes de tocar la red.
`strace -e trace=network ./cliente 127.0.01 5000`	`socket(...) = 3` y luego `IP invalida`	`socket` devolvió 3, el primer descriptor libre. Además reveló que **se creaba el socket antes de validar la IP**; se corrigió en la versión final.

Dos conceptos que dejó esta prueba:

- **Connection refused y Connection timed out no son lo mismo.** *Refused* significa que llegaste a la máquina y nadie escuchaba en ese puerto. *Timed out* significa que nadie respondió: un cortafuegos descartó tus paquetes en silencio o la máquina no existe o está apagada. Esta distinción fue la que permitió diagnosticar los problemas en AWS.

- **API frente a llamada al sistema.** En el código se escribe `recv`, pero `strace` muestra `recvfrom`. `recv` es una función de la biblioteca de C (glibc) y por dentro hace la llamada al sistema `recvfrom`. `strace` (*system call trace*) intercepta las llamadas al sistema, no las funciones de biblioteca.

Paso 4A: el protocolo

Para enviar números en vez de texto se definió un **protocolo binario de tamaño fijo**, escrito en `protocolo.h` y compartido por ambos programas. Resuelve tres problemas: TCP no respeta los límites de los mensajes, las máquinas pueden ordenar los bytes distinto, y no existe una función estándar para enviar un `double` por la red.

El contrato

Un **protocolo** es el acuerdo previo sobre qué bytes se envían, en qué orden y qué significan. Como los programas se compilan por separado y corren en máquinas distintas, es lo único que comparten.

Dirección	Contenido	Formato
Servidor → Cliente	`inicio` (8 bytes), luego `fin` (8 bytes)	`int64_t` en big-endian
Cliente → Servidor	suma parcial (8 bytes)	`double` IEEE 754 en big-endian

Por qué binario y no texto

	Texto (`"500000000"`)	Binario fijo (8 bytes)
Legible con `nc`	Sí	No
Tamaño del mensaje	Variable: hace falta un separador para saber dónde termina	Fijo: el receptor sabe que debe leer exactamente 8 bytes
Precisión del `double`	Se pierde si se imprimen pocos decimales	Exacta: viajan los 64 bits tal cual
Problema a resolver	Convertir a texto y volver a interpretarlo	El orden de bytes

Los tres problemas

1. TCP no respeta los límites de los mensajes. Un `send` de 8 bytes puede llegar en dos `recv` de 3 y 5. Además, `send` puede enviar **menos** bytes de los pedidos si el búfer de envío del kernel está lleno. Ambas funciones devuelven cuántos bytes movieron, y es responsabilidad del programa insistir hasta completar.

2. El orden de bytes con 64 bits. Es el mismo problema de `htons`, pero para 8 bytes. En Linux se usan `htobe64` (*host to big-endian 64*) y `be64toh` (el inverso), de la cabecera `<endian.h>`.

3. No existe `hton` para `double`. Un `double` son 64 bits en formato IEEE 754. La solución: copiar esos 64 bits, sin interpretarlos, a un `uint64_t`; convertir ese entero a big-endian; enviarlo. El receptor hace lo inverso. Esto supone que ambas máquinas usan IEEE 754, lo cual es cierto en cualquier EC2 x86 o ARM.

¿Y por qué no enviar directamente un `struct` con los dos números? Porque el compilador puede meter **relleno** (*padding*) entre campos y porque el orden de bytes depende de la máquina. Entre dos máquinas iguales funcionaría por casualidad, pero sería un protocolo frágil.

El código: `protocolo.h` (versión del paso 4)

```
#ifndef PROTOCOLO_H
#define PROTOCOLO_H

#include <stdint.h>      // int64_t, uint64_t
#include <string.h>      // memcpy
#include <endian.h>      // htobe64, be64toh
#include <sys/types.h>   // ssize_t, size_t
#include <sys/socket.h>  // send, recv

#define PUERTO_DEFECTO 5000

// Envía exactamente 'len' bytes.
// Devuelve 0 si se enviaron todos, -1 si hubo error.
static inline int enviar_todo(int fd, const void *datos, size_t len) {
    const char *p = datos;
    size_t enviados = 0;
    while (enviados < len) {
        ssize_t n = send(fd, p + enviados, len - enviados, 0);
        if (n < 0) return -1;
        enviados += (size_t)n;
    }
    return 0;
}
```

```
// Recibe exactamente 'len' bytes.
// Devuelve 1 si llegaron todos, 0 si el otro extremo cerró antes, -1 si hubo
error.
static inline int recibir_todo(int fd, void *datos, size_t len) {
    char *p = datos;
    size_t recibidos = 0;
    while (recibidos < len) {
        ssize_t n = recv(fd, p + recibidos, len - recibidos, 0);
        if (n < 0) return -1;
        if (n == 0) return 0;
        recibidos += (size_t)n;
    }
    return 1;
}

static inline int enviar_i64(int fd, int64_t valor) {
    uint64_t red = htobe64((uint64_t)valor);
    return enviar_todo(fd, &red, sizeof(red));
}

static inline int recibir_i64(int fd, int64_t *valor) {
    uint64_t red;
    int r = recibir_todo(fd, &red, sizeof(red));
    if (r == 1) *valor = (int64_t)be64toh(red);
    return r;
}

static inline int enviar_double(int fd, double valor) {
    uint64_t bits;
    memcpy(&bits, &valor, sizeof(bits));
    bits = htobe64(bits);
    return enviar_todo(fd, &bits, sizeof(bits));
}

static inline int recibir_double(int fd, double *valor) {
    uint64_t bits;
    int r = recibir_todo(fd, &bits, sizeof(bits));
    if (r == 1) {
        bits = be64toh(bits);
        memcpy(valor, &bits, sizeof(bits));
    }
    return r;
}

#endif
```

Línea por línea

Guarda de inclusión: **`#ifndef PROTOCOLO_H`**, **`#define PROTOCOLO_H`** y **`#endif`**. `#ifndef` (*if not defined*) significa "procesa lo que sigue solo si `PROTOCOLO_H` no está definido". La primera vez no lo está: se define y se incluye todo. Si por accidente la cabecera se incluye otra vez en el mismo `.c`, ya está definido y el preprocesador salta hasta `#endif`. Así no hay funciones definidas dos veces.

Cabeceras. `string.h` aporta `memcpy`; `endian.h`, las conversiones de 64 bits; `sys/types.h`, los tipos `size_t` y `ssize_t`.

`#define PUERTO_DEFECTO 5000`. El puerto vive en el contrato, así servidor y cliente no pueden quedar con valores distintos.

`static inline`. `static` hace que cada programa que incluya la cabecera tenga su propia copia privada de las funciones, sin choques al enlazar. `inline` sugiere al compilador insertar el cuerpo donde se llama y evita que `-Wall` advierta por las funciones que un programa no usa (el cliente nunca llama a `enviar_i64`).

`enviar_todo(int fd, const void *datos, size_t len)`

- Parámetros: el socket, un puntero a los datos y cuántos bytes enviar. `const void *` significa "puntero a cualquier tipo, que no voy a modificar"; así la función sirve para enteros, `double` o lo que sea.
- `const char *p = datos;` En C estándar no se puede hacer aritmética con `void *` (¿cuánto es "avanzar uno" si no se sabe el tamaño?). Viéndolo como `char *`, avanzar uno es avanzar exactamente un byte.
- `size_t enviados = 0;` cuenta cuántos bytes salieron ya.
- `while (enviados < len)` repite hasta enviarlo todo.
- `send(fd, p + enviados, len - enviados, 0)` envía **lo que falta**, empezando **donde quedó**. `p + enviados` es **aritmética de punteros**: la dirección `enviados` bytes después del inicio. Si de 8 bytes salieron 3, la siguiente vuelta intenta enviar 5 desde el byte 3.
- `if (n < 0) return -1;` error: se avisa a quien llamó.
- `enviados += (size_t)n;` `n` es `ssize_t` (con signo) y `enviados` es `size_t` (sin signo). Ya se sabe que `n` no es negativo, así que el *cast* es seguro y deja claro que se pensó.

`recibir_todo(int fd, void *datos, size_t len)` es el espejo de la anterior, con un caso extra: si `recv` devuelve 0, el otro proceso cerró la conexión **antes** de enviar todo lo prometido. Por eso devuelve tres valores: 1 (éxito), 0 (cierre prematuro) o -1 (error).

`enviar_i64` y **`recibir_i64`**

- `htobe64((uint64_t)valor)` convierte a big-endian. El *cast* a `uint64_t` se hace porque las funciones de orden de bytes trabajan con enteros sin signo; los bits no cambian.
- `enviar_todo(fd, &red, sizeof(red))` envía los 8 bytes de la variable `red`.
- En `recibir_i64`, `int64_t *valor` es un **puntero de salida**: la función escribe el resultado en la variable del que llama. `*valor = ...` significa "escribe en la variable a la que apunta `valor`" (operador de **desreferencia**). Solo se escribe si `r == 1`, es decir, si llegaron los 8 bytes.

`enviar_double` y `recibir_double`, y el `memcpy`

- `memcpy(&bits, &valor, sizeof(bits));` copia los 8 bytes del `double` en el entero `bits`, sin interpretarlos.
- Podría parecer más directo escribir `bits = *(uint64_t *)&valor;` ("lee la dirección de `valor` como si fuera un `uint64_t`"). En C eso es **comportamiento indefinido** por la regla de ***strict aliasing***: el compilador asume que un puntero a `double` y uno a `uint64_t` nunca apuntan al mismo lugar, y con `-O2` puede reordenar o eliminar lecturas basándose en esa suposición. `memcpy` es la forma correcta, y el compilador la reduce a una sola instrucción.

Prueba: ver el orden de bytes

```
#include <stdio.h>
#include "protocolo.h"

static void mostrar(const char *etiqueta, const void *ptr) {
    const unsigned char *b = ptr;
    printf("%-22s", etiqueta);
    for (int i = 0; i < 8; i++) printf("%02x ", b[i]);
    printf("\n");
}

int main(void) {
    int64_t x = 1000000000;                 // 0x3B9ACA00
    uint64_t x_red = htobe64((uint64_t)x);
    mostrar("int64 en memoria:", &x);
    mostrar("int64 para la red:", &x_red);

    double d = 0.7853981633974483;          // aprox. pi/4
    uint64_t bits;
    memcpy(&bits, &d, sizeof(bits));
    uint64_t bits_red = htobe64(bits);
```

```
    mostrar("double en memoria:", &d);
    mostrar("double para la red:", &bits_red);
    return 0;
}
```

- `#include "protocolo.h"` con comillas: busca la cabecera primero en la carpeta del proyecto, no en las del sistema.
- `mostrar` recorre los 8 bytes de cualquier objeto con un puntero `unsigned char *`. Mirar bytes con un puntero a `char` es la **excepción permitida** a la regla de *strict aliasing*.
- `%-22s` imprime la etiqueta alineada a la izquierda en 22 caracteres; `%02x` imprime cada byte en hexadecimal con dos dígitos.

Salida obtenida:

```
int64 en memoria:     00 ca 9a 3b 00 00 00 00
int64 para la red:    00 00 00 00 3b 9a ca 00
double en memoria:    18 2d 44 54 fb 21 e9 3f
double para la red:   3f e9 21 fb 54 44 2d 18
```

- 1 000 000 000 es `0x3B9ACA00`. En memoria aparece con el byte menos significativo primero (*little-endian*); en la versión para la red, en el orden en que se lee el número (*big-endian*).
- El orden de bytes **no cambia el valor**, solo cómo se acomoda en memoria.

El `double` leído con IEEE 754

En orden de red, los 8 bytes forman `0x3FE921FB54442D18`. Separándolo según el formato de doble precisión:

Campo	Bits	Valor	Significado
Signo	1	0	Positivo
Exponente	11	`0x3FE` = 1022	2 elevado a (1022 − 1023) = 2^{-1}
Mantisa	52	`0x921FB54442D18`	1,5707963… (con el "1," implícito)

- El 1023 que se resta es el **sesgo** del exponente: permite guardar exponentes negativos sin un bit de signo propio.
- El "1," de la mantisa no se guarda porque en un número normalizado siempre vale 1.

- El valor es 1,5707963… × 2^{-1} = 0,785398… = π/4. Como 1,5707963… es π/2, **π, π/2 y π/4 comparten la misma mantisa**: dividir por 2 en binario solo resta 1 al exponente. π se guarda como `40 09 21 fb 54 44 2d 18`, con exponente `0x400` = 1024, es decir 2^{1}.

Paso 4B: π entre dos procesos

En este paso `protocolo.h` se conecta al servidor y al cliente para calcular π de verdad entre dos procesos. Aquí se muestran las **versiones finales** de los tres archivos; las líneas de validación marcadas como "paso 5" se explican en la sección siguiente.

La decisión de diseño que permite el paralelismo

El servidor **envía el rango primero y calcula después**. Mientras él suma `[0, N/2)`, el cliente ya recibió su rango y suma `[N/2, N)` en otro núcleo o en otra máquina. Si el servidor calculara antes de enviar, el cliente pasaría todo ese tiempo bloqueado en `recv` y los cálculos ocurrirían en serie: el programa daría el mismo π, pero tardaría lo mismo que `leibniz` solo.

Organización de archivos

`serie.h` sabe de matemáticas y `protocolo.h` sabe de red; ninguno necesita saber del otro. Separar responsabilidades permite cambiar una sin tocar la otra.

El código: `serie.h`

```
#ifndef SERIE_H
#define SERIE_H

#include <stdint.h>   // int64_t
#include <time.h>     // clock_gettime, struct timespec

// Suma de Leibniz para k en [inicio, fin). NO multiplica por 4.
static inline double suma_parcial(int64_t inicio, int64_t fin) {
    double suma = 0.0;
    for (int64_t k = inicio; k < fin; k++) {
        double signo = (k % 2 == 0) ? 1.0 : -1.0;
        suma += signo / (2.0 * k + 1.0);
    }
    return suma;
}

// Segundos transcurridos desde un punto fijo, con resolución de
nanosegundos.
static inline double segundos_ahora(void) {
```

```
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

#endif
```

- `suma_parcial` es la misma función del paso 1, trasladada aquí para que la usen los dos programas.
- `time.h` declara `clock_gettime` y la estructura `struct timespec`.
- `struct timespec ts;` tiene dos campos: `tv_sec` (segundos enteros) y `tv_nsec` (nanosegundos restantes, de 0 a 999 999 999).
- `clock_gettime(CLOCK_MONOTONIC, &ts);` llena `ts` con la hora del reloj indicado. `CLOCK_MONOTONIC` es un reloj que **solo avanza**. El reloj normal (`CLOCK_REALTIME`) puede saltar hacia atrás cuando el sistema sincroniza la hora por red, y una medición podría dar un tiempo negativo. Para medir duraciones siempre se usa el monotónico.
- `ts.tv_sec + ts.tv_nsec / 1e9` combina ambos campos en un solo `double` de segundos. `1e9` es notación científica: 1×10^9.
- `clock_gettime` **casi nunca entra al kernel**: Linux mapea en cada proceso una pequeña región de código y datos del kernel (el vDSO) para leer la hora sin cambiar de modo. Por eso `strace` no la muestra.

El código: `servidor.c` (versión final)

```
#include <stdio.h>       // printf, fprintf, perror
#include <stdlib.h>      // exit, EXIT_FAILURE, strtoll, atoi
#include <string.h>      // memset
#include <unistd.h>      // close
#include <math.h>        // M_PI, fabs, isfinite
#include <arpa/inet.h>   // sockaddr_in, htons, htonl, ntohs, inet_ntop
#include <sys/socket.h>  // socket, setsockopt, bind, listen, accept
#include "protocolo.h"
#include "serie.h"

int main(int argc, char *argv[]) {
    // 0. Validar argumentos ANTES de pedir recursos al kernel
    if (argc < 2 || argc > 3) {
        fprintf(stderr, "Uso: %s <num_terminos> [puerto]\n", argv[0]);
        return 1;
    }
    char *resto;
    int64_t n = strtoll(argv[1], &resto, 10);
    if (*resto != '\0' || n <= 0) {
```

```
        fprintf(stderr, "num_terminos invalido: %s\n", argv[1]);
        return 1;
    }
    int puerto = (argc == 3) ? atoi(argv[2]) : PUERTO_DEFECTO;

    // 1. Crear el socket de escucha
    int fd_escucha = socket(AF_INET, SOCK_STREAM, 0);
    if (fd_escucha < 0) { perror("socket"); exit(EXIT_FAILURE); }

    // 2. Permitir reutilizar el puerto de inmediato
    int si = 1;
    if (setsockopt(fd_escucha, SOL_SOCKET, SO_REUSEADDR, &si, sizeof(si)) <
0) {
        perror("setsockopt"); exit(EXIT_FAILURE);
    }

    // 3. Construir la dirección donde vamos a escuchar
    struct sockaddr_in dir;
    memset(&dir, 0, sizeof(dir));
    dir.sin_family = AF_INET;
    dir.sin_addr.s_addr = htonl(INADDR_ANY);
    dir.sin_port = htons(puerto);

    // 4. Asociar el socket a esa dirección
    if (bind(fd_escucha, (struct sockaddr *)&dir, sizeof(dir)) < 0) {
        perror("bind"); exit(EXIT_FAILURE);
    }

    // 5. Convertirlo en socket pasivo
    if (listen(fd_escucha, 1) < 0) {
        perror("listen"); exit(EXIT_FAILURE);
    }
    printf("Coordinador escuchando en el puerto %d (N = %lld)\n", puerto,
(long long)n);

    // 6. Esperar al trabajador
    struct sockaddr_in dir_cliente;
    socklen_t tam = sizeof(dir_cliente);
    int fd_cliente = accept(fd_escucha, (struct sockaddr *)&dir_cliente,
&tam);
    if (fd_cliente < 0) { perror("accept"); exit(EXIT_FAILURE); }

    char ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &dir_cliente.sin_addr, ip, sizeof(ip));
    printf("Trabajador conectado desde %s:%d\n", ip,
ntohs(dir_cliente.sin_port));
```

```
    // 7. Repartir: el trabajador se queda con [mitad, n)
    double t_inicio = segundos_ahora();
    int64_t mitad = n / 2;
    if (enviar_i64(fd_cliente, mitad) < 0 || enviar_i64(fd_cliente, n) < 0) {
        perror("send"); exit(EXIT_FAILURE);
    }

    // 8. Calcular la mitad propia MIENTRAS el trabajador calcula la suya
    double t_calc = segundos_ahora();
    double suma_a = suma_parcial(0, mitad);
    double t_espera = segundos_ahora();

    // 9. Esperar el resultado del trabajador
    double suma_b;
    int r = recibir_double(fd_cliente, &suma_b);
    if (r < 0) { perror("recv"); exit(EXIT_FAILURE); }
    if (r == 0) {
        fprintf(stderr, "El trabajador cerro la conexion sin enviar su
resultado\n");
        exit(EXIT_FAILURE);
    }
    if (!isfinite(suma_b) || fabs(suma_b) > 1.0) {          // paso 5
        fprintf(stderr, "Suma invalida recibida del trabajador: %g\n",
suma_b);
        exit(EXIT_FAILURE);
    }
    double t_fin = segundos_ahora();

    // 10. Combinar y reportar
    double pi = 4.0 * (suma_a + suma_b);
    printf("pi    ~= %.15f\n", pi);
    printf("error  = %.3e\n", fabs(pi - M_PI));
    printf("calculo propio: %.3f s | espera al trabajador: %.3f s | total:
%.3f s\n",
           t_espera - t_calc, t_fin - t_espera, t_fin - t_inicio);

    close(fd_cliente);
    close(fd_escucha);
    return 0;
}
```

Lo nuevo del servidor, línea por línea

Los pasos 1 a 6 son los mismos del servidor mínimo. Lo nuevo:

Cabeceras. `math.h` aporta `M_PI` (π con precisión de `double`), `fabs` (valor absoluto de un `double`) e `isfinite`. `"protocolo.h"` y `"serie.h"` van entre comillas porque son del proyecto.

0. Validación de argumentos, antes de pedir recursos al kernel.

- `argc < 2 || argc > 3`: se acepta `./servidor N` o `./servidor N puerto`. `||` es el "o" lógico.
- `char *resto;` declara un puntero que `strtoll` llenará.
- `strtoll(argv[1], &resto, 10)`: esta vez se le pasa `&resto`, y `strtoll` deja en `resto` la dirección del **primer carácter que no pudo convertir**. Con `"1000"`, `resto` apunta al terminador `'\0'` (convirtió todo). Con `"10abc"`, convierte 10 y `resto` apunta a `"abc"`.
- `*resto != '\0'` significa "sobró texto": el argumento no era un número limpio. `n <= 0` descarta cero y negativos.
- `int puerto = (argc == 3) ? atoi(argv[2]) : PUERTO_DEFECTO;` usa el puerto dado o el del contrato.
- Validar antes de `socket()` evita pedir recursos al kernel que luego no se usarán (la mejora detectada con `strace` en el paso 3).

`(long long)n` con `%lld`. El tipo real detrás de `int64_t` cambia según la plataforma (`long` en Linux de 64 bits, `long long` en otras), y `printf` necesita que el formato coincida exactamente con el tipo. Se convierte a un tipo conocido y se usa su formato.

7. Repartir.

- `t_inicio` marca el momento en que empieza el trabajo distribuido.
- `int64_t mitad = n / 2;` el servidor se queda `[0, mitad)` y envía `[mitad, n)`.
- `enviar_i64(fd_cliente, mitad) < 0 || enviar_i64(fd_cliente, n) < 0`: el `||` de C evalúa en **cortocircuito**: si el primer envío falla, el segundo ni se intenta. Un solo `if` cubre los dos.

8. Calcular la mitad propia. `t_calc` y `t_espera` encierran el cálculo del servidor. Mientras se ejecuta este ciclo, el cliente ejecuta el suyo en paralelo.

9. Esperar el resultado.

- `recibir_double(fd_cliente, &suma_b)` bloquea al servidor hasta que lleguen los 8 bytes, si aún no han llegado.
- Se tratan por separado los tres resultados posibles: -1 (error de red), 0 (el trabajador se fue sin responder) y 1 (éxito). Son problemas distintos y conviene saber cuál ocurrió.
- La validación con `isfinite` se explica en el paso 5.

10. Reportar.

- `fabs(pi - M_PI)` es el error absoluto contra el π de la biblioteca. `%.3e` lo imprime en notación científica con 3 decimales.
- **cálculo propio** (`t_espera - t_calc`): CPU pura del servidor.
- **espera al trabajador** (`t_fin - t_espera`): cuánto estuvo bloqueado el servidor esperando la suma. Cerca de cero significa que el trabajador terminó antes o a la vez.
- **total** (`t_fin - t_inicio`): todo el trabajo distribuido.

El código: `cliente.c` (versión final)

```
#include <stdio.h>       // printf, fprintf, perror
#include <stdlib.h>      // exit, EXIT_FAILURE, atoi
#include <string.h>      // memset
#include <unistd.h>      // close
#include <arpa/inet.h>   // sockaddr_in, htons, inet_pton
#include <sys/socket.h>  // socket, connect
#include "protocolo.h"
#include "serie.h"

int main(int argc, char *argv[]) {
    if (argc < 2 || argc > 3) {
        fprintf(stderr, "Uso: %s <ip_servidor> [puerto]\n", argv[0]);
        return 1;
    }

    // 1. Construir y validar la dirección ANTES de crear el socket
    struct sockaddr_in dir;
    memset(&dir, 0, sizeof(dir));
    dir.sin_family = AF_INET;
    dir.sin_port = htons((argc == 3) ? atoi(argv[2]) : PUERTO_DEFECTO);
    if (inet_pton(AF_INET, argv[1], &dir.sin_addr) != 1) {
        fprintf(stderr, "IP invalida: %s\n", argv[1]);
        return 1;
    }

    // 2. Crear el socket y conectarse
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
    if (connect(fd, (struct sockaddr *)&dir, sizeof(dir)) < 0) {
        perror("connect"); exit(EXIT_FAILURE);
    }
    printf("Conectado al coordinador %s\n", argv[1]);

    // 3. Recibir y validar el rango asignado
    int64_t inicio, fin;
```

```
    if (recibir_i64(fd, &inicio) != 1 || recibir_i64(fd, &fin) != 1) {
        fprintf(stderr, "No se pudo recibir el rango completo\n");
        exit(EXIT_FAILURE);
    }
    if (inicio < 0 || fin <= inicio) {                          // paso 5
        fprintf(stderr, "Rango invalido recibido: [%lld, %lld)\n",
                (long long)inicio, (long long)fin);
        exit(EXIT_FAILURE);
    }
    printf("Rango asignado: [%lld, %lld)\n", (long long)inicio, (long
long)fin);

    // 4. Calcular la suma parcial
    double t0 = segundos_ahora();
    double suma = suma_parcial(inicio, fin);
    double t1 = segundos_ahora();
    printf("Suma parcial = %.17g (%.3f s)\n", suma, t1 - t0);

    // 5. Devolver el resultado y cerrar
    if (enviar_double(fd, suma) < 0) {
        perror("send"); exit(EXIT_FAILURE);
    }
    close(fd);
    return 0;
}
```

Lo nuevo del cliente, línea por línea

- **Argumentos.** El puerto ahora es opcional: `./cliente IP` usa el 5000 del contrato.
- **1. Validar antes de crear el socket.** La dirección se arma y se valida primero; si la IP es inválida, el programa sale sin haber pedido ningún descriptor al kernel.
- **3.** `int64_t inicio, fin;` declara las dos variables donde llegará el rango. El cliente se **bloquea** en `recibir_i64` hasta que lleguen los 16 bytes. `!= 1` cubre a la vez el error (-1) y el cierre prematuro (0).
- **El cliente no sabe nada de N ni de mitades.** Solo recibe un rango y lo calcula. Toda la decisión de cómo repartir vive en el servidor; por eso el diseño escala: con más trabajadores, el cliente no cambiaría.
- **4.** `t0` **y** `t1` miden el cálculo. `%.17g` imprime el `double` con 17 dígitos significativos, los necesarios para distinguir cualquier par de `double` distintos. `%g` elige entre notación normal y científica según convenga.
- **5.** Envía la suma y cierra. Al cerrar, el kernel avisa al otro extremo con un paquete FIN.

Compilar y ejecutar en local

```
gcc -Wall -Wextra -O2 -o servidor servidor.c -lm
gcc -Wall -Wextra -O2 -o cliente cliente.c
./servidor 10000000000          # terminal 1
./cliente 127.0.0.1             # terminal 2
```

- `-lm` enlaza la biblioteca matemática (`libm`), donde vive `fabs`. Va al final porque el enlazador resuelve las bibliotecas en el orden en que aparecen, después del código que las usa.
- El cliente no necesita `-lm` porque no usa funciones de `math.h`.
- **El servidor debe arrancar primero**: sin nadie escuchando, el cliente recibe `Connection refused`.

Los tiempos obtenidos y su análisis están en la sección de resultados.

Paso 5: ¿qué puede fallar?

El programa funcionaba cuando todo salía bien; este paso provoca fallos a propósito, observa la reacción y corrige lo necesario. Una revisión presencial suele atacar justo eso: "¿qué pasa si el otro proceso muere?", "¿qué pasa si llegan datos basura?".

Los cuatro experimentos

Experimento	Qué pasa	Por qué
1. `Ctrl+C` en el **cliente** a mitad del cálculo	El servidor termina su mitad e imprime `El trabajador cerro la conexion sin enviar su resultado`	`Ctrl+C` envía `SIGINT`, que termina el proceso. Al morir, el kernel cierra todos sus descriptores, y cerrar un socket TCP envía un FIN al otro extremo. El `recv` del servidor devuelve 0 y `recibir_todo` devuelve 0.
2. `Ctrl+C` en el **servidor** a mitad del cálculo	El cliente termina, "envía" su suma y sale **sin ningún error**	`send` con éxito **no significa que el otro proceso recibió los datos**: solo que el kernel los copió a su búfer de envío. El rechazo de la otra máquina llega cuando `send` ya regresó.

Experimento	Qué pasa	Por qué
3. `nc 127.0.0.1 5000` como cliente falso, escribiendo `holamundo`	El servidor obtiene un número absurdo	Esperaba 8 bytes y le llegaron 10. Los primeros 8 (`holamund`) se interpretan como los bits de un `double`. El protocolo binario no puede saber que no eran un número.
4. `nc -l 5000` como servidor falso, escribiendo 16 letras	El cliente recibe un rango enorme y se queda calculando indefinidamente	Igual que antes: 16 bytes de letras interpretados como dos `int64_t`. `-l` (*listen*) hace que `nc` escuche como un servidor.

Lección del experimento 2: con este protocolo, el cliente no puede saber si el servidor recibió su resultado. Para saberlo haría falta que el servidor respondiera con una **confirmación**. Es una buena pregunta de revisión.

Lección de los experimentos 3 y 4: nunca se debe confiar en datos que llegan por la red. Un error del otro proceso, un programa equivocado conectado al puerto o alguien malintencionado pueden enviar cualquier cosa.

Corrección 1: `MSG_NOSIGNAL` y `EINTR` en `protocolo.h`

Se agrega `#include <errno.h>` y se reemplazan los dos ciclos:

```
static inline int enviar_todo(int fd, const void *datos, size_t len) {
    const char *p = datos;
    size_t enviados = 0;
    while (enviados < len) {
        ssize_t n = send(fd, p + enviados, len - enviados, MSG_NOSIGNAL);
        if (n < 0) {
            if (errno == EINTR) continue;   // una señal interrumpió:
reintentar
            return -1;
        }
        enviados += (size_t)n;
    }
    return 0;
}

static inline int recibir_todo(int fd, void *datos, size_t len) {
    char *p = datos;
    size_t recibidos = 0;
```

```
    while (recibidos < len) {
        ssize_t n = recv(fd, p + recibidos, len - recibidos, 0);
        if (n < 0) {
            if (errno == EINTR) continue;   // una señal interrumpió:
reintentar
            return -1;
        }
        if (n == 0) return 0;
        recibidos += (size_t)n;
    }
    return 1;
}
```

- **`MSG_NOSIGNAL`**: el último argumento de `send` son *flags* (opciones). Si un proceso escribe en una conexión que el otro extremo ya cerró (por ejemplo, un segundo `send` después de un rechazo), el kernel le envía la señal **`SIGPIPE`**, cuya acción por defecto es **terminar el proceso en silencio**, sin ningún mensaje. Con `MSG_NOSIGNAL`, en vez de la señal, `send` devuelve -1 con `errno = EPIPE`: el error llega por el camino normal y `perror` lo imprime. Nuestro cliente solo hace un `send`, así que en el experimento 2 no cambia nada; protege a cualquier protocolo con más mensajes.
- **`EINTR`**: si un proceso **bloqueado** en `send` o `recv` recibe una señal que no lo mata, el kernel lo despierta para atenderla y la llamada regresa con -1 y `errno = EINTR` (interrumpida). No es un error real: `continue` salta al inicio del ciclo y se reintenta. Hoy los programas no capturan señales, pero la función queda correcta si mañana se agrega un manejador.
- **`errno` se revisa inmediatamente** después del fallo, porque la siguiente llamada que falle la sobrescribe.

Corrección 2: validar el rango en `cliente.c`

```
if (inicio < 0 || fin <= inicio) {
    fprintf(stderr, "Rango invalido recibido: [%lld, %lld)\n",
            (long long)inicio, (long long)fin);
    exit(EXIT_FAILURE);
}
```

El rango debe cumplir dos condiciones: el índice inicial no puede ser negativo (la serie empieza en k = 0), y el rango no puede estar vacío ni invertido.

Corrección 3: validar la suma en `servidor.c`

```
if (!isfinite(suma_b) || fabs(suma_b) > 1.0) {
    fprintf(stderr, "Suma invalida recibida del trabajador: %g\n", suma_b);
    exit(EXIT_FAILURE);
}
```

- `isfinite` es falso si el valor es infinito o NaN. Muchas combinaciones de bytes basura dan uno de los dos. `!` es el "no" lógico.
- `fabs(suma_b) > 1.0`: cualquier tramo de la serie de Leibniz suma, en valor absoluto, como mucho 1 (el primer término). Un valor mayor es imposible y delata datos corruptos. Esta comprobación usa **conocimiento del problema**, no de la red.
- Al verificar el código final con `holamundo`, el servidor respondió `Suma invalida recibida del trabajador: 1.14694e+195` y terminó con error.

Estas validaciones descartan los casos absurdos, pero no garantizan que un dato sea correcto: 8 bytes basura podrían caer por casualidad en un valor razonable. Una garantía real exigiría autenticar al otro proceso, algo fuera del alcance de la práctica.

Lo que no se corrigió: los tiempos de espera

Si la máquina del trabajador se apaga de golpe o se corta la red, nadie envía el aviso de cierre y el servidor queda bloqueado en `recv` **para siempre**. La opción de socket `SO_RCVTIMEO` (*receive timeout*) pone un límite a esa espera, pero exige elegir un tiempo mayor que lo que tarda el cálculo, que depende de N. Queda como mejora opcional.

Paso 6: llevarlo a AWS

El mismo código se ejecutó en dos instancias EC2: el coordinador en una y el trabajador en otra, comunicados por sus IP privadas. Sin cambiar ni una línea de C, solo la IP que recibe el cliente.

Conceptos

Una instancia EC2 es una máquina virtual. AWS divide un servidor físico entre varios clientes mediante un **hipervisor** (el software que crea y aísla máquinas virtuales), y cada instancia ejecuta **su propio kernel de Linux completo**. Son dos computadores independientes, con sus propios procesos, su propia tabla de descriptores y su propio planificador: el escenario ideal para procesos no emparentados, que no comparten memoria ni sistema de archivos.

Cada instancia tiene dos IP, y no son iguales.

	IP privada (ej. `172.31.0.27`)	IP pública (ej. `18.222.166.121`)
¿Quién la ve?	Solo las máquinas de tu red privada en AWS (la VPC)	Todo internet
¿Aparece en la interfaz de red de la instancia?	Sí	**No**: AWS la traduce desde afuera (NAT)
¿Cambia al detener y volver a iniciar?	No	Sí
¿Para qué se usa?	Cliente → servidor	Tu portátil → instancia, por SSH

Por eso el servidor usa `INADDR_ANY`: hacer `bind` a la IP pública fallaría, porque esa IP no existe dentro de la instancia.

El grupo de seguridad es un cortafuegos. Por defecto **bloquea todo lo que entra**. Se agregan reglas del tipo "permite TCP al puerto X desde el origen Y". Es **con estado**: si una conexión entrante está permitida, sus respuestas salen solas. Analogía: el portero de un edificio con tarjeta de acceso.

El par de claves reemplaza la contraseña. AWS guarda la clave pública dentro de la instancia y tú descargas la privada (`so-pi.pem`). SSH demuestra que tienes la privada sin enviarla nunca. AWS no permite descargarla otra vez.

Costos. Se usó `t3.micro` (2 CPU virtuales, 1 GiB de memoria), que está dentro de la capa gratuita. Se evitaron las `t4g`, que usan procesadores ARM: el código funcionaría, pero se cambiaría de arquitectura sin necesidad. **Detener** una instancia es apagarla (se puede volver a encender); **terminar** es borrarla para siempre, con su disco. Al terminar cada sesión se detienen; al terminar la práctica se terminan.

Bloque 1: el grupo de seguridad, antes que las instancias

El primer intento creó el grupo desde el asistente de lanzamiento y falló (ver solución de problemas). El orden que funcionó fue crear primero el grupo con sus dos reglas:

1. Elegir una región arriba a la derecha y no cambiarla: los grupos de seguridad pertenecen a una región.
2. Averiguar la IP pública propia con `curl -4 ifconfig.me`. Ese sitio devuelve la dirección desde la que le llegó la petición, que es la misma que verá AWS. `-4` fuerza IPv4.

3. EC2 → Grupos de seguridad → Crear grupo de seguridad. Nombre `so-pi-sg`. Regla de entrada: tipo SSH (puerto 22), origen personalizado `TU_IP/32`.
4. Editar las reglas de entrada del grupo ya creado y agregar: TCP personalizado, puerto 5000, origen `so-pi-sg` (el mismo grupo). Esta regla no se puede poner al crear el grupo porque aún no existe para referirse a sí mismo.

Qué significa `/32`. Las reglas aceptan bloques de direcciones en notación CIDR. Una IPv4 tiene 32 bits, y el número después de la barra dice cuántos quedan fijos. Con `/32` quedan fijos todos, así que el bloque contiene una sola dirección. `0.0.0.0/0` no fija ningún bit: todo internet.

Por qué el puerto 5000 se abre al propio grupo. La regla significa "permite conexiones al 5000 desde cualquier instancia que pertenezca a este grupo". Como ambas instancias están en él, el trabajador llega al coordinador, y nadie más en internet puede enviar basura al servidor.

Bloque 2: las instancias

EC2 → Lanzar instancias: nombre `coordinador`; imagen Ubuntu Server 24.04 LTS, 64 bits (x86), la misma familia que el portátil; tipo `t3.micro`; nuevo par de claves `so-pi` (RSA, `.pem`); en red, **seleccionar el grupo existente** `so-pi-sg`; número de instancias: 2. Luego se renombra la segunda como `trabajador` y se espera a que ambas digan "En ejecución".

Bloque 3: entrar por SSH

```
mkdir -p ~/.ssh
mv ~/Descargas/so-pi.pem ~/.ssh/
chmod 400 ~/.ssh/so-pi.pem
ssh -i ~/.ssh/so-pi.pem ubuntu@IP_PUBLICA_DEL_COORDINADOR
ssh -i ~/.ssh/so-pi.pem ubuntu@IP_PUBLICA_DEL_TRABAJADOR   # en otra terminal
```

- `mkdir -p` crea la carpeta `~/.ssh` si no existe (`-p` evita el error si ya existe). `~` es tu carpeta personal.
- `chmod 400` deja el archivo como `r--------`: solo el dueño puede leerlo y nadie puede escribirlo. Los permisos en octal son tres cifras (dueño, grupo, otros) y cada una suma lectura (4), escritura (2) y ejecución (1). SSH **se niega a usar** una llave privada que otros puedan leer (`UNPROTECTED PRIVATE KEY FILE`).
- `ssh -i` indica qué llave usar (*identity*). `ubuntu` es el usuario predeterminado de las imágenes de Ubuntu en AWS. La primera vez se responde `yes` para confiar en la huella de la máquina.

- Dentro, el prompt cambia a `ubuntu@ip-172-31-0-27:~$`, que ya contiene la IP privada. `hostname -I` la imprime: `172.31.0.27` en el coordinador y `172.31.11.87` en el trabajador.

Bloque 4: compilar y ejecutar en la nube

```
# En las dos instancias
sudo apt update
sudo apt install -y build-essential

# En tu portátil, dentro de la carpeta del proyecto
scp -i ~/.ssh/so-pi.pem protocolo.h serie.h servidor.c leibniz.c
ubuntu@IP_PUBLICA_DEL_COORDINADOR:~/
scp -i ~/.ssh/so-pi.pem protocolo.h serie.h cliente.c
ubuntu@IP_PUBLICA_DEL_TRABAJADOR:~/

# En el coordinador
gcc -Wall -Wextra -O2 -o servidor servidor.c -lm
gcc -Wall -Wextra -O2 -o leibniz leibniz.c
./servidor 10000000000

# En el trabajador
gcc -Wall -Wextra -O2 -o cliente cliente.c
./cliente 172.31.0.27
```

- Las imágenes de Ubuntu en AWS vienen sin compilador. `apt update` descarga la lista actualizada de paquetes; `apt install -y build-essential` instala `gcc`, `make` y las cabeceras de la biblioteca de C. `-y` responde "sí" a la confirmación.
- `scp` copia archivos a otra máquina a través de SSH, cifrados, con la misma llave. `ubuntu@IP:~/` significa "en esa máquina, como `ubuntu`, en su carpeta personal". Se ejecuta en el portátil, **no** dentro de las instancias.
- Se copia el código fuente y no los ejecutables porque un ejecutable depende de la arquitectura del procesador y de las versiones de las bibliotecas de la máquina donde se compiló.
- El cliente usa la **IP privada** del coordinador: viaja por la red interna de AWS y pasa la regla del puerto 5000.
- El servidor imprimió `Trabajador conectado desde 172.31.11.87:60648`: la IP privada del trabajador, prueba de que eran dos máquinas distintas.

Solución de problemas: lo que pasó y cómo se resolvió

Síntoma	Causa	Arreglo
`ssh` se queda con el cursor parpadeando y termina en `Connection timed out`	El grupo de seguridad descarta los paquetes en silencio: la IP de casa cambió desde el día anterior, o la instancia no quedó con el grupo correcto	Comparar `curl -4 ifconfig.me` con la regla SSH; revisar en la pestaña Seguridad de la instancia qué grupo tiene
No aparece la opción "Mi IP"	Es solo un atajo de la consola	Escribir la IP a mano como `IP/32`
La instancia no tiene `so-pi-sg`	Al lanzar no se seleccionó el grupo existente	Acciones → Seguridad → Cambiar grupos de seguridad, en ambas instancias
`so-pi-sg` no aparece al buscarlo	Se creó en otra región	Crearlo en la región de las instancias
`Permission denied (publickey)`	Usuario o llave equivocados	Usar `ubuntu@` y la ruta correcta de `so-pi.pem`
`UNPROTECTED PRIVATE KEY FILE`	Permisos de la llave muy abiertos	`chmod 400`
El cliente recibe `Connection refused`	El servidor no está corriendo o la IP es otra	Arrancar primero el servidor; usar la IP privada del coordinador
El cliente se queda esperando en `connect`	Falta la regla del puerto 5000 o una instancia no está en el grupo	Revisar la regla y el grupo de ambas

El método de diagnóstico fue separar tramos con `nc -zv -w 5 HOST PUERTO` (`-z` solo comprueba si el puerto acepta conexión, `-v` muestra el resultado, `-w 5` se rinde a los 5 segundos):

1. `nc -zv -w 5 github.com 22` respondió `succeeded`: la red de casa sí deja salir el puerto 22.
2. `nc -zv -w 5 18.222.166.121 22` agotó el tiempo con `Operation now in progress` (el texto de `EINPROGRESS`: la conexión seguía "en curso" cuando `nc` se rindió). Conclusión: los paquetes llegaban a AWS y el grupo de seguridad los descartaba.
3. Tras corregir el grupo, el mismo comando respondió `succeeded` y SSH funcionó.

`nc` prueba solo el camino de red, sin llaves ni usuarios, así que si falla el problema es de red y no de SSH.

Si la IP de casa cambia de nuevo, SSH vuelve a agotar el tiempo: se actualiza la regla con la nueva `IP/32`. La alternativa es abrir SSH a `0.0.0.0/0`; sigue protegido por la llave (las imágenes de Ubuntu en AWS no aceptan contraseñas), pero expone el puerto 22 a los robots que recorren internet. Aceptable para una práctica corta, no para algo que quede encendido.

Resultados y análisis

En AWS, con $N = 10^{10}$, la versión distribuida tardó 6,62 s frente a 13,26 s de un solo proceso: una aceleración de 2,0 con dos máquinas, la máxima posible. Las tres ejecuciones dieron el mismo π, `3.141592653488201`, con error $1{,}016 \times 10^{-10}$.

Tiempos medidos ($N = 10^{10}$)

Escenario	Un solo proceso (s)	Mitad del servidor (s)	Mitad del cliente (s)	Total distribuido (s)
Portátil con batería	—	22,03	12,08	22,03
Portátil conectado	8,83	9,19	4,49	9,19
AWS, dos t3.micro	13,26	6,62	6,54	6,62

Aceleración y eficiencia

Dos medidas estándar comparan una versión paralela con la secuencial:

```
S = \frac{T_{\text{1 proceso}}}{T_{\text{distribuido}}} = \frac{13{,}26}{6{,}62} \approx 2{,}0 \qquad E = \frac{S}{\text{número de procesos}} = \frac{2{,}0}{2} \approx 100\,\%
```

Se logra el máximo porque **casi todo el programa es paralelizable y la comunicación es despreciable**: viajan 16 bytes de ida y 8 de vuelta, frente a más de 6 segundos de cálculo. Si cada término necesitara un mensaje, la red dominaría el tiempo y la aceleración se desplomaría. La pregunta clave de cualquier problema distribuido es cuánto se calcula por cada byte que se comunica.

Lo que explica cada observación

- **La energía importa.** Con batería, Ubuntu y el procesador bajan la frecuencia para ahorrar energía, y los tiempos se multiplicaron. Toda comparación debe hacerse en las mismas condiciones.
- **En el portátil, el servidor tardó el doble que el cliente** aunque ejecuta la misma función con la misma cantidad de términos. La versión distribuida (9,19 s) resultó más lenta que la de un proceso (8,83 s), porque el total depende del proceso más lento. El código no lo explica; la causa está en **dónde ubicó el planificador cada proceso**: núcleos de distinto tipo, dos CPU lógicas del mismo núcleo físico o la frecuencia. No se verificó cuál de estas fue. En AWS, cada proceso con su propia máquina, los tiempos quedaron parejos (6,62 frente a 6,54 s), lo que respalda esa explicación.
- **"Espera al trabajador: 0,000 s".** El trabajador terminó unos 0,08 s antes y envió su suma mientras el coordinador seguía calculando. Esos 8 bytes esperaron en el **búfer de recepción del socket, dentro del kernel del coordinador**. Cuando el servidor llamó a `recv`, los datos ya estaban ahí y la llamada regresó sin bloquear. El kernel recibe los paquetes por su cuenta; el proceso solo los recoge cuando quiere.
- **El mismo π bit a bit.** `leibniz.c` ya calculaba por separado `[0, mitad)` y `[mitad, n)`: son exactamente las mismas operaciones en el mismo orden, así que los bits coinciden aunque ocurran en dos computadores. Funciona porque el protocolo transporta el `double` completo y en un orden de bytes acordado.
- **La t3.micro es más lenta que el portátil** (13,26 s frente a 8,83 s con un proceso): cada CPU virtual de esta instancia rinde menos que un núcleo del portátil. Su `time` mostró `user` 13,257 s y `sys` 0,001 s: el mismo perfil limitado por CPU del primer día.

Por qué la suma del cliente es diminuta

El cliente devolvió `2.4999999999769033e-11`, mientras que la mitad del servidor vale casi 0,785. No es un error. Agrupando los términos de dos en dos, para k par:

```
\frac{1}{2k+1} - \frac{1}{2k+3} = \frac{2}{(2k+1)(2k+3)} \approx \frac{1}
{2k^2}
```

Cada par es positivo y muy pequeño cuando k es grande. Sumando desde $M = 5 \times 10^9$ hasta $N = 10^{10}$:

```
\text{suma}_B \approx \frac{1}{4}\left(\frac{1}{M} - \frac{1}{N}\right) =
\frac{1}{4}\left(2 \times 10^{-10} - 10^{-10}\right) = 2{,}5 \times 10^{-11}
```

Las dos mitades tienen **el mismo trabajo** (5 000 millones de términos cada una) pero **no la misma contribución**: los primeros términos construyen casi todo π y la segunda mitad solo pule el décimo dígito. Es otra forma de ver por qué la serie converge tan lento.

Referencia de comandos de Linux

Todos los comandos usados en la práctica, con lo que muestran y el concepto de SO que permiten observar.

Comando	Qué hace	Concepto que muestra
`gcc -Wall -Wextra -O2 -o prog prog.c -lm`	Compila con advertencias, optimización y la biblioteca matemática	Del código fuente al ejecutable
`time ./prog`	Mide tiempo real, de usuario y de sistema	Modo usuario frente a modo kernel; procesos limitados por CPU o por E/S
`echo $?`	Muestra el código de salida del último programa	0 éxito, otro valor error
`ss -tlnp`	Lista los sockets TCP que escuchan y su proceso	Sockets en estado `LISTEN`
`ss -tan`	Lista todas las conexiones TCP con su estado	Estados como `TIME-WAIT`
`ps -o pid,stat,wchan:25,cmd -C servidor`	Muestra PID, estado y dónde espera el proceso	Estados de proceso: `R` ejecutando, `S` bloqueado
`ps -o pid,psr,stat,pcpu,cmd -C servidor,cliente`	Añade la CPU donde corre y el uso de CPU	Ubicación de procesos por el planificador
`watch -n 1 'comando'`	Repite un comando cada segundo	Observar procesos mientras viven
`ls -l /proc/<PID>/fd`	Lista los descriptores abiertos de un proceso	La tabla de descriptores

Comando	Qué hace	Concepto que muestra
`cat /proc/sys/net/ipv4/ip_local_port_range`	Muestra el rango de puertos efímeros	Puertos asignados por el kernel
`pgrep servidor`	Devuelve el PID de un proceso por su nombre	Identificación de procesos
`kill <PID>`	Envía `SIGTERM` al proceso	Señales; el kernel cierra los descriptores al morir el proceso
`nc IP PUERTO`	Se conecta como cliente TCP	Probar un servidor sin escribir código
`nc -l PUERTO`	Escucha como servidor TCP	Probar un cliente sin escribir código
`nc -zv -w 5 IP PUERTO`	Solo comprueba si el puerto acepta conexiones	Diagnosticar tramos de red
`strace -e trace=network ./prog`	Muestra las llamadas al sistema de red	API (`recv`) frente a llamada al sistema (`recvfrom`)
`lscpu` / `nproc`	Describe el procesador / cuenta las CPU	Hardware disponible para el planificador
`hostname -I`	Muestra las IP de la máquina	IP privada de una instancia
`ip -4 addr show`	Muestra las interfaces de red IPv4	La IP pública de AWS no aparece: NAT
`curl -4 ifconfig.me`	Muestra tu IP pública	La dirección que ven los demás
`chmod 400 archivo`	Deja el archivo solo de lectura para el dueño	Permisos de archivos

Comando	Qué hace	Concepto que muestra
`rm -f archivo`	Borra sin preguntar, aunque sea de solo lectura	Borrar depende de los permisos de la carpeta
`ssh -i llave.pem ubuntu@IP`	Abre una terminal remota	Acceso a otra máquina
`scp -i llave.pem archivos ubuntu@IP:~/`	Copia archivos a otra máquina	Transferencia cifrada por SSH
`sudo apt update` / `sudo apt install -y build-essential`	Actualiza la lista de paquetes e instala el compilador	Gestión de software con privilegios

Código final completo

Estos cuatro archivos son la versión de entrega, con todas las correcciones del paso 5. Se compilaron con `-Wall -Wextra` sin ninguna advertencia y se probaron de nuevo el 14 de octubre de 2026: calculan π entre dos procesos y rechazan una suma basura enviada con `nc`.

```
gcc -Wall -Wextra -O2 -o servidor servidor.c -lm
gcc -Wall -Wextra -O2 -o cliente cliente.c
./servidor <num_terminos> [puerto]
./cliente <ip_servidor> [puerto]
```

`protocolo.h`

```
#ifndef PROTOCOLO_H
#define PROTOCOLO_H

#include <stdint.h>      // int64_t, uint64_t
#include <string.h>      // memcpy
#include <errno.h>       // errno, EINTR
#include <endian.h>      // htobe64, be64toh
#include <sys/types.h>   // ssize_t, size_t
#include <sys/socket.h>  // send, recv, MSG_NOSIGNAL

#define PUERTO_DEFECTO 5000

// Envía exactamente 'len' bytes.
```

```
// Devuelve 0 si se enviaron todos, -1 si hubo error.
static inline int enviar_todo(int fd, const void *datos, size_t len) {
    const char *p = datos;
    size_t enviados = 0;
    while (enviados < len) {
        ssize_t n = send(fd, p + enviados, len - enviados, MSG_NOSIGNAL);
        if (n < 0) {
            if (errno == EINTR) continue;   // una señal interrumpió:
reintentar
            return -1;
        }
        enviados += (size_t)n;
    }
    return 0;
}

// Recibe exactamente 'len' bytes.
// Devuelve 1 si llegaron todos, 0 si el otro extremo cerró antes, -1 si hubo
error.
static inline int recibir_todo(int fd, void *datos, size_t len) {
    char *p = datos;
    size_t recibidos = 0;
    while (recibidos < len) {
        ssize_t n = recv(fd, p + recibidos, len - recibidos, 0);
        if (n < 0) {
            if (errno == EINTR) continue;   // una señal interrumpió:
reintentar
            return -1;
        }
        if (n == 0) return 0;
        recibidos += (size_t)n;
    }
    return 1;
}

static inline int enviar_i64(int fd, int64_t valor) {
    uint64_t red = htobe64((uint64_t)valor);
    return enviar_todo(fd, &red, sizeof(red));
}

static inline int recibir_i64(int fd, int64_t *valor) {
    uint64_t red;
    int r = recibir_todo(fd, &red, sizeof(red));
    if (r == 1) *valor = (int64_t)be64toh(red);
    return r;
}
```

```
static inline int enviar_double(int fd, double valor) {
    uint64_t bits;
    memcpy(&bits, &valor, sizeof(bits));
    bits = htobe64(bits);
    return enviar_todo(fd, &bits, sizeof(bits));
}

static inline int recibir_double(int fd, double *valor) {
    uint64_t bits;
    int r = recibir_todo(fd, &bits, sizeof(bits));
    if (r == 1) {
        bits = be64toh(bits);
        memcpy(valor, &bits, sizeof(bits));
    }
    return r;
}

#endif
```

`serie.h`

```
#ifndef SERIE_H
#define SERIE_H

#include <stdint.h>   // int64_t
#include <time.h>     // clock_gettime, struct timespec

// Suma de Leibniz para k en [inicio, fin). NO multiplica por 4.
static inline double suma_parcial(int64_t inicio, int64_t fin) {
    double suma = 0.0;
    for (int64_t k = inicio; k < fin; k++) {
        double signo = (k % 2 == 0) ? 1.0 : -1.0;
        suma += signo / (2.0 * k + 1.0);
    }
    return suma;
}

// Segundos transcurridos desde un punto fijo, con resolución de
nanosegundos.
static inline double segundos_ahora(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
```

```
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

#endif
```

servidor.c

```
#include <stdio.h>       // printf, fprintf, perror
#include <stdlib.h>      // exit, EXIT_FAILURE, strtoll, atoi
#include <string.h>      // memset
#include <unistd.h>      // close
#include <math.h>        // M_PI, fabs, isfinite
#include <arpa/inet.h>   // sockaddr_in, htons, htonl, ntohs, inet_ntop
#include <sys/socket.h>  // socket, setsockopt, bind, listen, accept
#include "protocolo.h"
#include "serie.h"

int main(int argc, char *argv[]) {
    // 0. Validar argumentos ANTES de pedir recursos al kernel
    if (argc < 2 || argc > 3) {
        fprintf(stderr, "Uso: %s <num_terminos> [puerto]\n", argv[0]);
        return 1;
    }
    char *resto;
    int64_t n = strtoll(argv[1], &resto, 10);
    if (*resto != '\0' || n <= 0) {
        fprintf(stderr, "num_terminos invalido: %s\n", argv[1]);
        return 1;
    }
    int puerto = (argc == 3) ? atoi(argv[2]) : PUERTO_DEFECTO;

    // 1. Crear el socket de escucha
    int fd_escucha = socket(AF_INET, SOCK_STREAM, 0);
    if (fd_escucha < 0) { perror("socket"); exit(EXIT_FAILURE); }

    // 2. Permitir reutilizar el puerto de inmediato
    int si = 1;
    if (setsockopt(fd_escucha, SOL_SOCKET, SO_REUSEADDR, &si, sizeof(si)) <
0) {
        perror("setsockopt"); exit(EXIT_FAILURE);
    }

    // 3. Construir la dirección donde vamos a escuchar
    struct sockaddr_in dir;
    memset(&dir, 0, sizeof(dir));
    dir.sin_family = AF_INET;
```

```
    dir.sin_addr.s_addr = htonl(INADDR_ANY);
    dir.sin_port = htons(puerto);

    // 4. Asociar el socket a esa dirección
    if (bind(fd_escucha, (struct sockaddr *)&dir, sizeof(dir)) < 0) {
        perror("bind"); exit(EXIT_FAILURE);
    }

    // 5. Convertirlo en socket pasivo
    if (listen(fd_escucha, 1) < 0) {
        perror("listen"); exit(EXIT_FAILURE);
    }
    printf("Coordinador escuchando en el puerto %d (N = %lld)\n", puerto,
(long long)n);

    // 6. Esperar al trabajador
    struct sockaddr_in dir_cliente;
    socklen_t tam = sizeof(dir_cliente);
    int fd_cliente = accept(fd_escucha, (struct sockaddr *)&dir_cliente,
&tam);
    if (fd_cliente < 0) { perror("accept"); exit(EXIT_FAILURE); }

    char ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &dir_cliente.sin_addr, ip, sizeof(ip));
    printf("Trabajador conectado desde %s:%d\n", ip,
ntohs(dir_cliente.sin_port));

    // 7. Repartir: el trabajador se queda con [mitad, n)
    double t_inicio = segundos_ahora();
    int64_t mitad = n / 2;
    if (enviar_i64(fd_cliente, mitad) < 0 || enviar_i64(fd_cliente, n) < 0) {
        perror("send"); exit(EXIT_FAILURE);
    }

    // 8. Calcular la mitad propia MIENTRAS el trabajador calcula la suya
    double t_calc = segundos_ahora();
    double suma_a = suma_parcial(0, mitad);
    double t_espera = segundos_ahora();

    // 9. Esperar el resultado del trabajador
    double suma_b;
    int r = recibir_double(fd_cliente, &suma_b);
    if (r < 0) { perror("recv"); exit(EXIT_FAILURE); }
    if (r == 0) {
        fprintf(stderr, "El trabajador cerro la conexion sin enviar su
resultado\n");
```

```
        exit(EXIT_FAILURE);
    }
    if (!isfinite(suma_b) || fabs(suma_b) > 1.0) {
        fprintf(stderr, "Suma invalida recibida del trabajador: %g\n",
suma_b);
        exit(EXIT_FAILURE);
    }
    double t_fin = segundos_ahora();

    // 10. Combinar y reportar
    double pi = 4.0 * (suma_a + suma_b);
    printf("pi    ~= %.15f\n", pi);
    printf("error  = %.3e\n", fabs(pi - M_PI));
    printf("calculo propio: %.3f s | espera al trabajador: %.3f s | total:
%.3f s\n",
           t_espera - t_calc, t_fin - t_espera, t_fin - t_inicio);

    close(fd_cliente);
    close(fd_escucha);
    return 0;
}
```

cliente.c

```
#include <stdio.h>       // printf, fprintf, perror
#include <stdlib.h>      // exit, EXIT_FAILURE, atoi
#include <string.h>      // memset
#include <unistd.h>      // close
#include <arpa/inet.h>   // sockaddr_in, htons, inet_pton
#include <sys/socket.h>  // socket, connect
#include "protocolo.h"
#include "serie.h"

int main(int argc, char *argv[]) {
    if (argc < 2 || argc > 3) {
        fprintf(stderr, "Uso: %s <ip_servidor> [puerto]\n", argv[0]);
        return 1;
    }

    // 1. Construir y validar la dirección ANTES de crear el socket
    struct sockaddr_in dir;
    memset(&dir, 0, sizeof(dir));
    dir.sin_family = AF_INET;
    dir.sin_port = htons((argc == 3) ? atoi(argv[2]) : PUERTO_DEFECTO);
    if (inet_pton(AF_INET, argv[1], &dir.sin_addr) != 1) {
        fprintf(stderr, "IP invalida: %s\n", argv[1]);
```

```
        return 1;
    }

    // 2. Crear el socket y conectarse
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
    if (connect(fd, (struct sockaddr *)&dir, sizeof(dir)) < 0) {
        perror("connect"); exit(EXIT_FAILURE);
    }
    printf("Conectado al coordinador %s\n", argv[1]);

    // 3. Recibir y validar el rango asignado
    int64_t inicio, fin;
    if (recibir_i64(fd, &inicio) != 1 || recibir_i64(fd, &fin) != 1) {
        fprintf(stderr, "No se pudo recibir el rango completo\n");
        exit(EXIT_FAILURE);
    }
    if (inicio < 0 || fin <= inicio) {
        fprintf(stderr, "Rango invalido recibido: [%lld, %lld)\n",
                (long long)inicio, (long long)fin);
        exit(EXIT_FAILURE);
    }
    printf("Rango asignado: [%lld, %lld)\n", (long long)inicio, (long
long)fin);

    // 4. Calcular la suma parcial
    double t0 = segundos_ahora();
    double suma = suma_parcial(inicio, fin);
    double t1 = segundos_ahora();
    printf("Suma parcial = %.17g (%.3f s)\n", suma, t1 - t0);

    // 5. Devolver el resultado y cerrar
    if (enviar_double(fd, suma) < 0) {
        perror("send"); exit(EXIT_FAILURE);
    }
    close(fd);
    return 0;
}
```